Man vs Nature vs Buffer: Building a Truly Random Bank

Jack "Siggy" Sigler
Department of Computer Science
Colorado School of Mines
Golden, CO
sigler@mines.edu

Noah Chapman
Department of Computer Science
Colorado School of Mines
Golden, CO
nchapman@mines.edu

Zachary Samuels
Department of Computer Science
Colorado School of Mines
Golden, CO
zsamuels@mines.edu

Trevor Sheehy
Department of Computer Science
Colorado School of Mines
Golden, CO
Tsheehy@mines.edu

Dawson Matthews
Department of Computer Science
Colorado School of Mines
Golden, CO
dawsonmatthews@mines.edu

Sam Abderholden
Department of Computer Science
Colorado School of Mines
Golden, CO
sabderholden@mines.edu

***Abstract*—This paper goes over the building of a true random number generator (TRNG) that uses real world audio and video from Clear Creek in Golden, Colorado along with /dev/urandom. We used a Raspberry Pi to collect this data which mixes these sources of entropy using hashing and XOR to create one output of bits that can be used to pass the NIST randomness suite to confirm its statistical randomness. Finally, we outline current security guarantees, known limitations, and future work towards a centrally managed network of these devices that can serve clients on demand.**

Keywords—true random number generator, distributed entropy

I. Introduction

Reliable cryptography depends on unpredictable, random numbers. Servers and other headless computers with little user interaction can fall short, as their built-in entropy pools may not refill quickly enough. To boost robustness, we combine three distinct entropy sources with significant noise: system entropy from /dev/urandom, environmental sound captured by a microphone, and video frames of Clear Creek. Each stream is hashed to a fixed length, then XOR-mixed to cancel bias in any one feed. After validating this approach on a single machine, we deployed a Raspberry Pi on the creek to capture the entropy and output a stream of statistically random bytes [2].

II. Background

Entropy quality and amount is central to any TRNG to provide sufficient randomness. System interfaces like /dev/urandom perform well on machines with GUIs and user manipulation by effectively using the user's own randomness in inputs to generate entropy, however this style of entropy gathering can stagnate on headless hardware such as servers. To combat this, the use of external entropy sources can be used as supplemental sources of randomness. Audio contributes randomness through artifacts like microphone hissing and electrical fluctuations alongside the actual sound recorded. Video can also offer dense entropy where every subtle shift in lighting or movement can affect thousands of pixels to produce a high volume of data that is hard to replicate. Prior efforts to use these sources such as Cloudflare's lava-lamp wall show the value of this type of entropy collection. By using hashes and XOR functions, we can increase the overall unpredictability, and as long as one source remains chaotic at any one point, the final output will have enough entropy and remain secure.

III. Stages of Development

A. Initial Entropy Collection and Byte Generation

Objective

The primary objective of Stage 1 was to develop a prototype script capable of gathering entropy from multiple sources—specifically, audio, video, and system-level data—and using this entropy to generate a random byte stream of a user-specified length.

Entropy Collection

- System Entropy: Entropy was collected from system-level sources such as /dev/urandom, which is a widely used interface for pseudo-random data on Unix-based systems.

- Audio Entropy: Live audio data was captured through the computer's microphone, introducing real-world variability and noise.
- Video Entropy: For video-based entropy, a YouTube livestream of a fish tank was used as a proof-of-concept source. This enabled validation of the video entropy capture functionality before transitioning to more dynamic or real-time environmental sources.

Entropy Processing and Combination

To ensure consistency across entropy sources, each stream of raw entropy was first processed using the SHA3-256 hashing algorithm, yielding fixed-length outputs. The resulting hash values from each source were then combined using a bitwise XOR operation. This technique increases the overall entropy quality by blending independent entropy streams.

Random Byte Output

The final product of this stage was a continuously generated stream of random bytes, written to a binary file. A producer-consumer model was implemented in the script: the producer continuously gathered and processed entropy into the binary file, while the consumer accessed this data for downstream applications or further testing.

B. Distributed Entropy Collection With Raspberry Pis

Objective

The second stage builds upon the initial prototype by introducing a distributed entropy collection framework. This involves deploying one or more Raspberry Pi devices to collect environmental entropy (e.g., audio and video) from various physical locations such as Clear Creek.

Raspberry Pi Setup

Each Raspberry Pi unit is equipped with a webcam to capture both audio and video feeds. The lightweight nature and flexibility of the Raspberry Pi platform make it well-suited for remote deployment and continuous entropy harvesting.

Distributed Entropy Collection

Each Raspberry Pi operates an instance of the entropy collection script locally. These scripts collect, hash, and XOR raw entropy data to generate random byte sequences on the device itself.

Secure Data Transmission

To maintain data integrity and privacy, the entropy data or processed output is securely transmitted to a machine. Secure protocols such as SSH or other encrypted communication methods are used to protect data in transit.

Real-Time Entropy Generation

This distributed setup allows for continuous, real-time entropy harvesting from multiple, independent physical environments. These contributions help supply diverse and high-quality entropy to the TRNG system, enhancing its reliability and robustness.

IV. System Breakdown

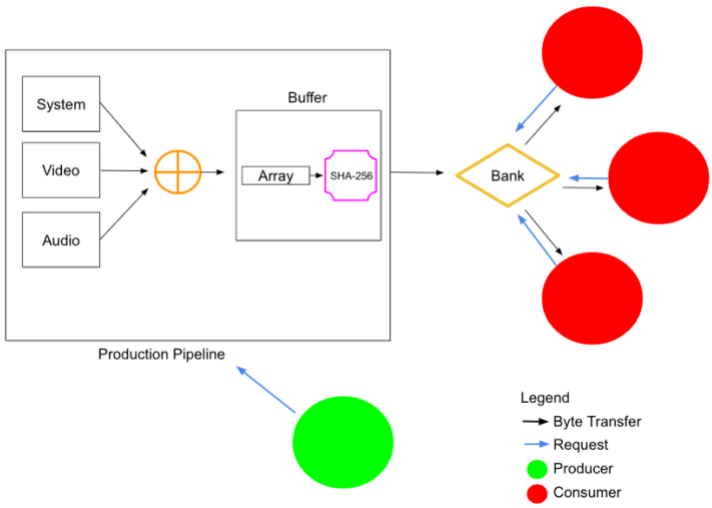

Fig. 1 Overall System Breakdown

A. High Level Overview

This system generates cryptographically secure random numbers by collecting entropy from real-world sources and managing their storage and retrieval through a buffered producer-consumer architecture as pictured in Figure 1. It is designed to run on a Raspberry Pi and leverages environmental noise, including system entropy as well as audio and video captured from a dedicated webcam setup overlooking Clear Creek in Golden, Colorado, to ensure high-quality randomness.

Entropy collection is handled in collectors.py, which sources 32 bytes each from three independent streams: system entropy (os.urandom), audio input, and live video frames. These three sources are then XORed together byte-wise to produce a single 32-byte combined entropy sample. This redundancy increases resilience against potential biases or weaknesses in any one entropy stream.

The combined entropy samples are passed to the entropy buffer implemented in buffer.py. This buffer accumulates raw entropy in a bytearray. Once the total number of bytes reaches a predefined flush threshold (32 bytes), the buffer is flushed by computing a SHA3-

256 digest of its contents. This results in a cryptographically mixed 32-byte output, which is then cleared from the buffer to make room for new entropy.

The digests produced by the entropy buffer are written to a persistent storage file, referred to as the entropy bank (random_bank.bin), via the bank.py module. The bank uses file locking (portalocker) to enforce mutual exclusion between producers and consumers accessing the file. This ensures data consistency and prevents race conditions during concurrent read/write operations. The bank adds new entropy bytes to the end of the bank file, and when they are consumed they are taken from the front of the bank file.

The producer module (producer.py) operates in a continuous loop, collecting entropy, buffering it, and writing flushed digests to the bank. The producer captures video frames using OpenCV from a webcam alongside audio and system entropy. It continues running until manually interrupted, at which point it performs a final buffer flush to ensure no entropy is lost.

The consumer module (consumer.py) periodically attempts to retrieve 32-byte chunks from the entropy bank. If sufficient data is available, the bank returns the requested bytes and rewrites the file without the consumed portion. File access is again guarded by exclusive locks to prevent conflicts with the producer. The consumer loop is customizable via parameters such as read interval and chunk size, and outputs retrieved entropy in a hex-encoded format.

The system is designed to be extensible. While currently implemented with a single shared bank file and simple XOR-based entropy mixing, future improvements may include rotating chunk files, multi-threaded producers, and entropy quality monitoring using NIST SP 800-22 tests.

B. *System Entropy*

Computer system entropy is the most basic type of random data collected for this project. While people may think of system entropy from sources like /dev/urandom and /dev/random on Linux systems to be unconditionally secure after boot time, that is not always the case. The quality of entropy provided by these system files is very dependent on the system and the hardware available to it. In our research of system entropy, we learned about types of systems that can lack entropy for sufficient true randomness, as well as changes that have been made to the random and urandom protocols that alter how those sources should be used safely.

In order for a computer system to generate entropy on its own, they use information from hardware peripherals such as keyboard input timings, mouse movement data as well as hardware interrupts from other devices. These inputs tend to be sufficiently unpredictable on systems with consistent input from a user. As long as a user continues to type sentences, navigate their system with their mouse, or use other connected devices with the computer, the entropy will be high enough to produce sufficiently random data for encryption. However, while this works well for personal computers with daily use, it becomes problematic for headless servers, systems on chips (SoC), or other devices without much user input. These devices have fewer sources of entropy and thus are more prone to producing predictable data. Furthermore, this issue is worsened by recent changes to the Linux random files.

In our research on system entropy, we learned that in the past it was commonly recommended to use /dev/urandom for general random number generation, and /dev/random for highly confidential secrets. This is because in the past, /dev/random used to block requests if the estimated system entropy was below a set threshold to ensure proper random number generation. However, this distinction was changed in 2020 with the release of Linux 5.6, and now both /dev/random as well as /dev/urandom do not block requests, even if system entropy is low. This change could easily be causing users to generate insufficiently random numbers without knowing their system's entropy is too low, especially considering the documentation provided by the man random command still claims /dev/random will block on lack of entropy. This oversight is certainly a cryptographic security risk and should be fixed through updating documentation.

While /dev/random and /dev/urandom are good pseudo random number generators overall, they also have weaknesses and oversights that must be acknowledged, especially in systems lacking user input. Due to these flaws, we have decided to introduce two additional sources of entropy into our random number generator to increase the cryptographic security of our solution.

C. *Sound Entropy*

The audio source of entropy, while at first glance providing enough nondeterminism, has clear limitations and drawbacks that can significantly alter its implementation. Quiet environments, or spaces where there are only a few human voices, provide low entropy and/or contain easily discernible patterns that have the potential to limit the effectiveness and obscurity of a TRNG. These circumstances, however, were accounted for and tested in Stage 1 and were not

present during Stage 2, resulting in an independently effective source of entropy for the Clear Creek TRNG.

The workflow for generating sufficiently random keys was straightforward. Audio was first collected using the Pyaudio library in Python 3 using the maximum standard sampling rate, for most built-in microphones, of 44.1kHz. Of note is that a sufficiently high sample rate, combined with the frequency response of the microphone are crucial to capturing randomness, as most of the entropy lies in subtle variations. This is especially true in an environment which produces mostly white (randomly distributed) noise, such as the rushing water in Clear Creek. After recording for a set duration of 0.5 seconds, the sample (recorded as a .wav file) was amplified to increase detail, as well as ADC noise, before being hashed with SHA3-256. Using SHA3 served to both increase nondeterminism, even with relatively low entropy audio sources, while compressing the large audio stream into 256 bits. The Pyaudio library was chosen for its simplicity, flexibility, and speed (a critical part of this section of the TRNG). Audio amplification was not originally part of the workflow for this section but was necessary for Stage 1.

Our initial findings when testing the audio subsystem showed that it failed to provide sufficient randomness, as it was unable to pass many of the NIST tests for TRNGs. As shown in Figure 2, audio samples taken in a quiet environment exhibited a narrow frequency spectrum dominated by a 120Hz hum, highlighting the lack of entropy in low-noise conditions. This limitation stems from a core weakness of audio-based entropy: quiet environments introduce low variability and often contain easily discernible patterns. In particular, mains voltage introduces a 120Hz signal in the microphone via ground loops, producing oscillations in the electronics—much like an antenna. Even applying SHA3-256 hashing was insufficient to mask this structure or produce NIST-compliant random output.

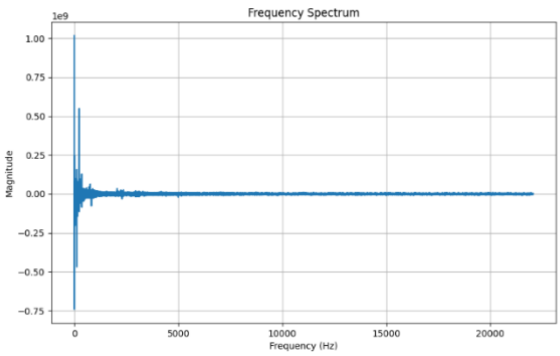

Fig. 2 The Frequency Spectrum of an Audio Sample Taken in a Quiet Room

This problem was solved by increasing the gain of the microphone which accomplished two things: making the frequency spectrum more discernible and increasing the entropy gathered from electrical noise in the ADC. Microphones need to convert analog audio signals into digital ones, which provide two more sources of entropy, thermal noise and quantization noise, both of which are increased when the audio is amplified. The system becomes more responsive to vibration/noise, but less stable, adding chaos and making it less predictable. This effectively mitigated the noise constraint in quiet environments, resulting in the system passing all NIST tests for Stage1.

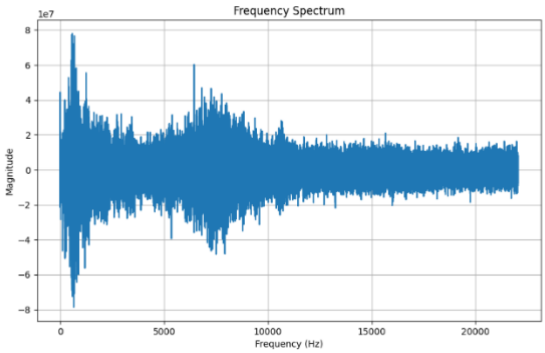

Fig. 3 Frequency Spectrum from Audio Sample Recorded at Clear Creek

While there were significant variables to account for in Stage 1, these were not present in Stage 2, so little to no audio amplification was required to meet NIST standards. Recording at Clear Creek, we observed a healthy distribution of frequencies across the spectrum that the microphone could detect. Figure 3 demonstrates the broader and more uniform frequency distribution observed when sampling audio at Clear Creek, confirming the site's effectiveness as a high-entropy environment. Specifically, there was a normal distribution within the 5kHz to 10kHz range, with higher frequencies appearing uniformly distributed. Overall, we found Clear Creek to be an excellent source of audio-only entropy, though we retained the amplification code for use during periods when the creek may be frozen or running shallow, conditions which would reduce its effectiveness.

D. *Video Entropy*

In both Stage 1 and Stage 2 of our TRNG development, video data played a key role as one of the primary sources of entropy. In Stage 1, we implemented a method that periodically pulled individual frames from a fishtank YouTube livestream as the entropy input. Each frame was flattened into a one-dimensional array of pixel values, which was then

hashed using SHA3-256 to produce a fixed-length byte sequence as seen in figure 4. In Stage 2, this approach was adapted to use a live webcam feed connected to Raspberry Pi devices, allowing for real-time entropy generation from the surrounding environment (shown in figure 5).

Why Video is a Strong Source of Entropy

Video streams, especially those capturing real-world, naturally dynamic scenes (such as a fish tank or a flowing river), are excellent candidates for entropy for several reasons:

- High Variability in Pixel Data: Every video frame contains thousands (or millions) of pixel values. Even slight movements like a fish swimming by, a ripple in water, or changes in lighting can cause significant, unpredictable shifts in pixel data. This results in a large and constantly changing entropy space. An example of this is how Cloudflare uses a video stream of lava lamps as a source of entropy, as the constantly changing patterns result in good entropy.
- Temporal Unpredictability: Because video is time-based, any frame pulled from the stream is influenced not just by the scene itself but also by the exact moment it was captured. The timing of frame capture adds another layer of unpredictability, especially when paired with natural variability in frame rate or environmental lighting.
- Environmental Factors: Real-world scenes introduce entropy from physical phenomena that are inherently difficult to model or predict such as: fluid motion, reflections, shadows, or biological movement. These sources are highly resistant to replication, unlike purely software-based pseudorandom functions.
- Noise and Compression: Livestreams often include noise introduced by the video encoding process or network transmission, further contributing to entropy in subtle and non-reproducible ways. On top of that, in our stage 2 implementation using a poor quality / noisy video source like the webcam introduces extra entropy to the system.

By using video from natural, continuously changing environments we ensured that the collected data was highly unpredictable and resistant to reverse engineering. Flattening and hashing the pixel data of each frame before combining it with other entropy sources ensured a fixed-length, high-entropy input suitable for our random byte stream generation.

V. Security Analysis

From a systems-level perspective, this architecture is designed to reliably generate cryptographically secure random numbers by aggregating entropy from multiple independent, real-world sources and enforcing robust post-processing and access control mechanisms.

At its foundation, the system adheres to the core security principles expected of a true random number generator (TRNG): unpredictability, forward secrecy, and output integrity. By combining entropy from the operating system with audio and video signals sourced from natural environmental variation at Clear Creek in Golden, Colorado, the system reduces reliance on any single entropy stream. This design minimizes the risk of predictability or compromise in the overall output.

Entropy from each source is aggregated through an XOR operation, a well-established method for mixing independent randomness. This ensures that as long as *at least one* input source remains secure and unpredictable, the combined result preserves cryptographic entropy. The output of this mixing step is then passed through SHA3-256, a cryptographic hash function known for its strong diffusion and collision resistance. This hashing stage not only compresses any bias or structural regularity from the raw input but also obscures any observable patterns that may exist in individual entropy sources.

The system further enhances security by incorporating an intermediate buffering stage. This buffer introduces temporal separation between entropy collection and output generation, preventing direct correlation between input and output over time. This obfuscation layer reinforces forward secrecy, making it substantially more difficult for an adversary to reconstruct the state of the system or predict future outputs even with knowledge of prior inputs.

Together, these mechanisms: source diversification, cryptographic mixing and hashing, temporal decoupling, and file-level locking—form a holistic architecture that upholds the requirements of a secure, scalable TRNG suitable for cryptographic applications.

To validate the statistical soundness of our output, we subjected the generated random bytes to the NIST SP 800-22 statistical test suite, a widely recognized benchmark for evaluating the quality of random number generators. Our output successfully passed all 15 tests, each of which targets a distinct property of randomness—such as frequency, runs, autocorrelation, and entropy compression resistance. Passing the full suite provides strong empirical evidence that the mixing and hashing approach described above effectively eliminates bias and preserves unpredictability. This serves as proof of concept that our system, under its current configuration, is capable of producing output

consistent with the expectations of cryptographic randomness. [4]

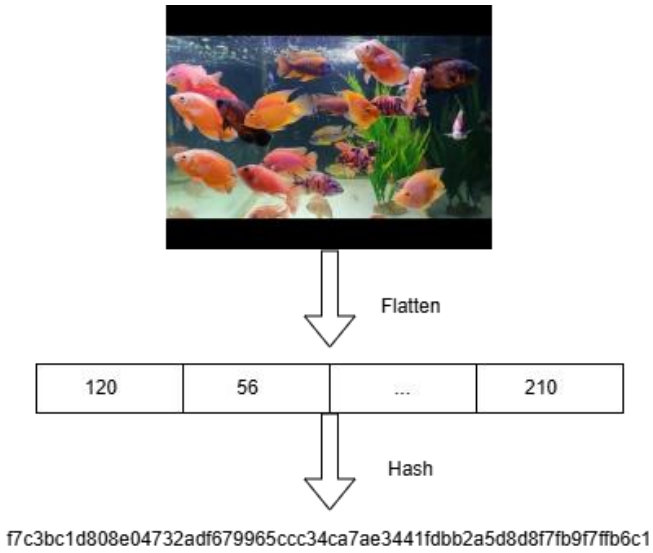

Fig. 4 Depiction of frame flattening process

Fig. 5 Scene recorded at Clear Creek

VI. Future Plans

As previously outlined, the current system has been designed with extensibility in mind. A key direction for future development involves scaling the architecture to support distributed entropy collection and centralized distribution.

In the envisioned expansion, a central server will be introduced to coordinate multiple producers and serve multiple authenticated consumers. Each producer will be a Raspberry Pi system deployed at distinct locations along Clear Creek in Golden, Colorado, each equipped with a webcam and microphone for local entropy collection. Once a producer accumulates a sufficient quantity of random bytes, it will transmit this data securely to the central server.

To efficiently handle the increased data load and mitigate the I/O overhead associated with frequent file access, the server will maintain a rotating carousel of fixed-size files, rather than a single monolithic entropy bank. The server will track the active file being consumed, append new entropy as it is received, and create new files as necessary to manage capacity.

This future architecture centralizes responsibility in the server, which must:

- Authenticate incoming data from producers and requests from consumers.
- Manage storage by rotating and indexing entropy files for efficient read/write operations.
- Facilitate access control by allowing authenticated users to request arbitrary amounts of random data.

This design enables robust scalability, modular deployment, and secure, on-demand access to high-quality random data from geographically distributed sources.

VII. Limitations

While the system demonstrates strong theoretical and empirical security properties, including successful validation through the NIST SP 800-22 test suite, it is important to recognize its current limitations from both architectural and operational standpoints.

First, the security of the system relies on the assumption that at least one entropy source remains uncompromised and truly unpredictable. Although the use of system, audio, and video entropy collected from a natural environment reduces the risk of uniform failure, coordinated manipulation or degradation of all sources could theoretically impact output quality. Additionally, while XOR and SHA3-256 are cryptographically sound for mixing, the system does not implement redundancy in its mixing approach, nor does it perform real-time health checks or entropy estimation beyond the initial NIST test results. This may limit its robustness in long-running deployments without active monitoring.

From a performance perspective, the current implementation uses a single entropy bank file with file-level locking to coordinate access between producer and consumer processes. Although functional at small scale, this introduces a potential I/O bottleneck and limits parallelism as the system expands. Furthermore, the design is presently constrained to a single producer-consumer loop on a single device. While future work includes distribution across multiple Raspberry Pi devices and centralized coordination, the current architecture does not yet

support authentication, secure communication, or file segmentation for high-throughput or concurrent access.

There are also practical concerns regarding the hardware environment. The webcam and microphone are exposed to their physical surroundings and could be influenced by environmental manipulation. Deliberate interferences—such as controlled light or sound injections—could degrade the quality or unpredictability of collected entropy. Additionally, the system lacks platform abstraction and is tightly coupled to the Raspberry Pi hardware and its peripheral dependencies (e.g., OpenCV, yt-dlp), limiting portability to other environments without modification.

Lastly, while the use of a buffer introduces temporal separation between entropy input and output, the system currently does not support internal reseeding or entropy pool maintenance beyond SHA3-256 hashing. This may limit long-term resilience compared to more mature TRNG frameworks like Fortuna or Yarrow, which incorporate periodic reseeding and adaptive entropy accumulation.

In recognizing these limitations, the current system serves as a robust and extensible foundation, with opportunities for enhancement in scalability, resilience, and monitoring as it evolves toward a production-grade entropy infrastructure.

VIII. Conclusion

This project successfully developed a true random number generator that collects entropy from natural sources-such as environmental audio, video and system noise-to produce cryptographically secure random binary data. By combining multiple independent data streams, followed by a SHA3-256 hash, the system works even if an entropy source is compromised. The system passed all NIST SP 800-22 tests validating our methodology and the effectiveness of our design. The system does not yet support internal reseeding, or fault tolerance, and could be affected by deliberate environmental interference. Future work can expand the system into a distributed network with a central server to authenticate and manage data, as well as provide a system for authenticated users to request as much data as they need. Overall, our project demonstrates the viability for a secure entropy generation system that utilizes natural phenomena for cryptographic infrastructure.

References

[1] R. DeGarmo, "True random vs. pseudorandom number generation," *wolfSSL*, Oct. 14, 2024. Available: wolfSSL.com/true-random-vs-pseudorandom-number-generation/

[2] How do lava lamps help with internet encryption? | Cloudflare. Available: cloudflare.com/learning/ssl/lava-lamp-encryption/. Accessed: Apr. 25, 2025.

[3] *Random bit generation: Documentation and software*, NIST. Available: csrc.nist.gov/projects/random-bit-generation/documentation-and-software

[4] *Random numbers generated from audio and video sources*, *ResearchGate*. Available: researchgate.net/publication/258391920_Random_Numbers_Generated_from_Audio_and_Video_Sources. Accessed: Apr. 29, 2025.